

# Relatório T3

MC906: Redes Neurais do Zero com Interpretabilidade

Adriano Ribeiro Franulovic Campos, Andrei Quaresma Pinto, Matheus Ferracciú Scatolin,  
Pietro Fernandes Magaldi, Tiago Perrupato Antunes

Maio/2026

## Resumo

Implementamos uma rede neural multicamadas (MLP) 100% *NumPy* para classificar tumores do dataset *Breast Cancer Wisconsin (Diagnostic)* em benignos ou malignos. O trabalho cobre forward pass, backpropagation e mini-batch SGD com ReLU, Softmax e cross-entropy; exploração de hiperparâmetros; avaliação final com early stopping; e interpretabilidade por gradientes de entrada e ablação. O melhor modelo alcançou **97,4%** de acurácia no conjunto de teste (115 amostras), com três erros.

## 1 Introdução

O diagnóstico precoce de câncer de mama depende de características morfológicas dos núcleos celulares extraídas de imagens histopatológicas. O dataset *Breast Cancer Wisconsin (Diagnostic)* [1] disponibiliza 569 amostras com 30 atributos numéricos por tumor (médias, piores valores e erros de medidas como raio, textura, concavidade etc.) e rótulo binário.

**Objetivo do TP:** construir do zero uma MLP capaz de classificar tumores, compreender o aprendizado via retropropagação e *explicar* as decisões do modelo. Não utilizamos PyTorch, TensorFlow ou bibliotecas equivalentes para o treinamento; apenas NumPy para a rede e scikit-learn para carregamento de dados, métricas auxiliares e comparação opcional.

**Pré-processamento.** Invertemos o encoding do scikit-learn para adotar B=0 (benigno) e M=1 (maligno). Aplicamos *split* estratificado 64/16/20% (363 treino, 91 validação, 115 teste), preservando a proporção de classes ( $\approx 62,7\%$  benignos). Cada feature recebe z-score com média e desvio *calculados apenas no treino*, evitando vazamento de informação. Para treino usamos *one-hot encoding* dos rótulos, compatível com Softmax + cross-entropy.

A Figura 1 mostra projeção PCA 2D do teste: as classes são parcialmente separáveis, com região de sobreposição onde erros são mais prováveis.

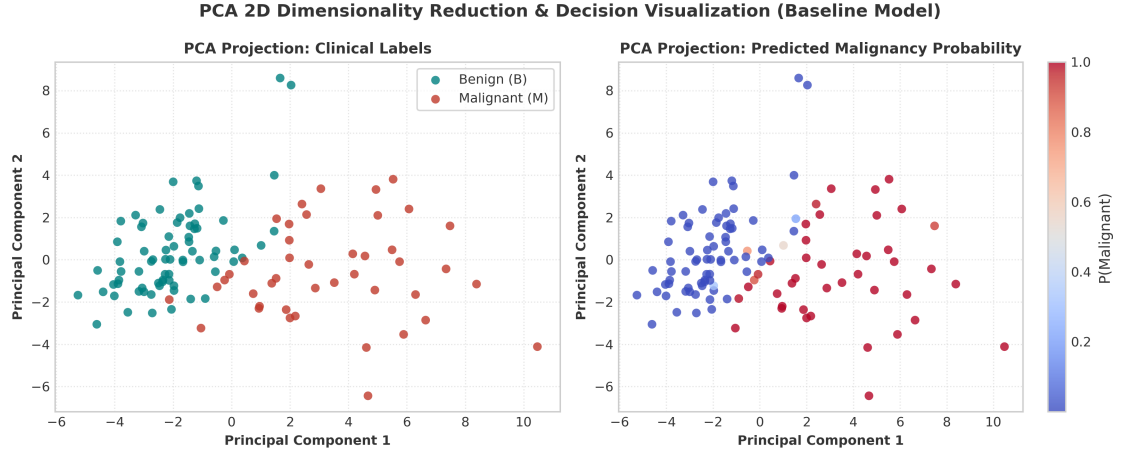


Figura 1: Projeção PCA 2D do conjunto de teste (esquerda: rótulos reais; direita:  $P(\text{maligno})$  do modelo *baseline*).

## 2 Implementação

### 2.1 Arquitetura

A rede é totalmente conectada com  $L$  camadas definidas por uma lista de tamanhos, e.g.  $[30, 16, 2]$ : 30 entradas, 16 neurônios ReLU na camada oculta e 2 logs de saída. Para cada camada  $l$ :

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \text{ReLU}(\mathbf{z}^{(l)}) \quad (l < L), \quad (1)$$

com  $\mathbf{a}^{(0)} = \mathbf{x}$ . Na saída aplicamos Softmax explícita:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}. \quad (2)$$

### 2.2 Função de custo e retropropagação

Usamos cross-entropy multiclasse com rótulos *one-hot*  $\mathbf{y}$ :

$$\mathcal{L} = -\frac{1}{B} \sum_{b=1}^B \sum_i y_{bi} \log p_{bi}. \quad (3)$$

Combinando Softmax + CE, o gradiente na saída simplifica-se a  $(\mathbf{p} - \mathbf{y})/B$ , propagado pelas camadas ocultas via regra da cadeia e gradiente da ReLU:  $\partial a / \partial z = \mathbb{I}[z > 0]$ .

### 2.3 Otimização

Parâmetros atualizados por mini-batch SGD; opcionalmente com **momentum**  $\mu$ :  $\mathbf{v} \leftarrow \mu \mathbf{v} - \alpha \nabla \mathcal{L}$ ,  $\theta \leftarrow \theta + \mathbf{v}$ . Implementamos também inicialização **He** para camadas ReLU ( $\mathcal{N}(0, \sqrt{2/n_{\text{in}}})$ ). Pesos padrão usam  $\mathcal{N}(0, 0.01)$  e vieses zero. Validamos a implementação com *gradient check* numérico (erro  $< 10^{-6}$ ).

## 3 Experimentos

### 3.1 Metodologia

**Métricas:** acurácia e loss de cross-entropy. Hiperparâmetros são selecionados pela **acurácia de validação** (média de 5 *seeds*); o teste é usado **uma única vez** após a escolha final (Seção 5). Em empates de validação, mantemos a configuração mais simples.

**Baseline** (Seção 3):  $\text{arch}=[30,32,2]$ ,  $\alpha = 0,1$ ,  $\text{batch}=32$ , 150 épocas, 3 *seeds*. Converge rapidamente ( $\approx 97$  a  $98\%$  val acc), com leve *overfitting* após a época 15 (val loss sobe enquanto treino continua caindo; ver notebook, Figura baseline).

### 3.2 Exploração de hiperparâmetros

Variamos **um hiperparâmetro por vez** (100 épocas, *seeds* 1, 2, 42, 100, 2026). A Tabela 1 resume os resultados.

Tabela 1: Exploração de hiperparâmetros (100 épocas, 5 *seeds*; varia-se um parâmetro por vez).

| Parâmetro | Valor        | Train acc | Val acc      | Val loss |
|-----------|--------------|-----------|--------------|----------|
| $\alpha$  | 0,001        | 0,628     | 0,626        | 0,658    |
|           | 0,01         | 0,986     | <b>0,978</b> | 0,064    |
|           | 0,1          | 1,000     | 0,967        | 0,009    |
| Batch     | 8            | 0,992     | <b>0,978</b> | 0,024    |
|           | 32           | 0,986     | 0,978        | 0,064    |
|           | 64           | 0,964     | 0,967        | 0,141    |
| Arch      | [30,16,2]    | 0,992     | <b>0,978</b> | 0,025    |
|           | [30,32,2]    | 0,992     | 0,978        | 0,024    |
|           | [30,64,32,2] | 0,995     | 0,978        | 0,027    |

**Melhor configuração:**  $\text{arch}=[30,16,2]$ ,  $\alpha = 0,01$ ,  $\text{batch}=8$  (val acc 0,978; escolhida por parsimônia ante empate).  $\alpha = 0,001$  permanece no patamar da classe majoritária ( $\approx 0,63$ ), indicando *underfitting* severo.

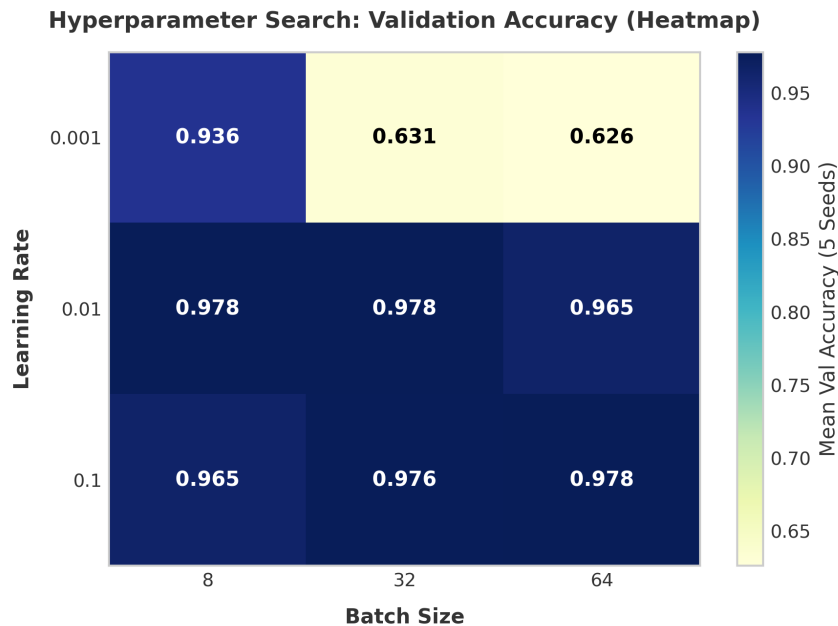


Figura 2: Heatmap de val acc ( $\alpha \times \text{batch}$ ).

A Figura 3 evidencia a **fase de latência**: curvas iniciam em  $\approx 62,7\%$  (357/569 benignos) enquanto pesos pequenos e ReLUs inativas impedem quebra de simetria. Latência:  $\alpha = 0,1 \approx 2$  épocas;  $\alpha = 0,01 \approx 18$ ;  $\alpha = 0,001$  não sai do platô; **batch**=8 mais rápido que 64; rede [30,64,32,2]  $\approx 45$  épocas.

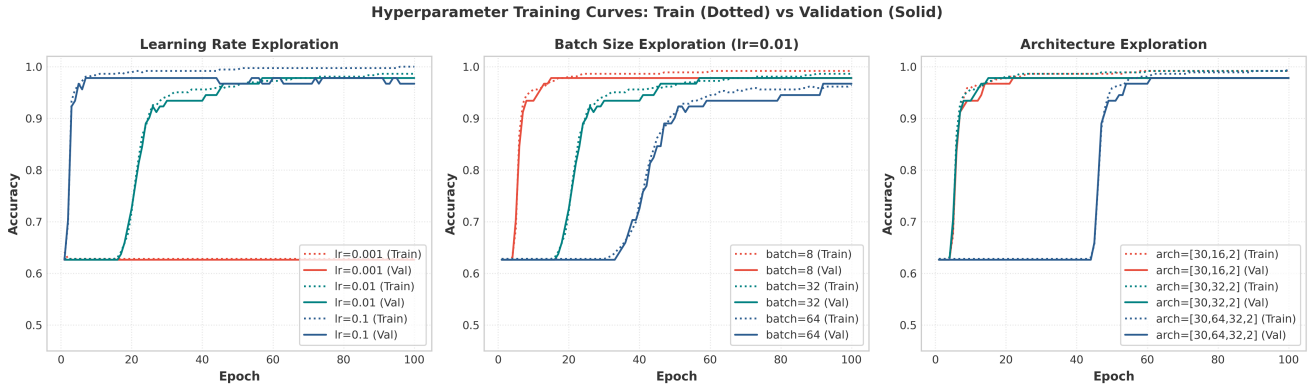


Figura 3: Curvas de acurácia ao variar  $\alpha$ , **batch** e arquitetura (treino pontilhado, validação sólida).

### 3.3 Treinamento final

Adotamos **early stopping** em duas fases: (1) treinar em treino+validação separados por 200 épocas e identificar a época de menor val loss (época 37, loss 0,108); (2) retreinar em treino+validação (454 amostras) por exatamente 37 épocas e avaliar o teste uma vez.

## 4 Resultados

**Teste final:** acurácia **97,4%** (112/115 corretos), test loss 0,085. A Figura 4 mostra a matriz de confusão.

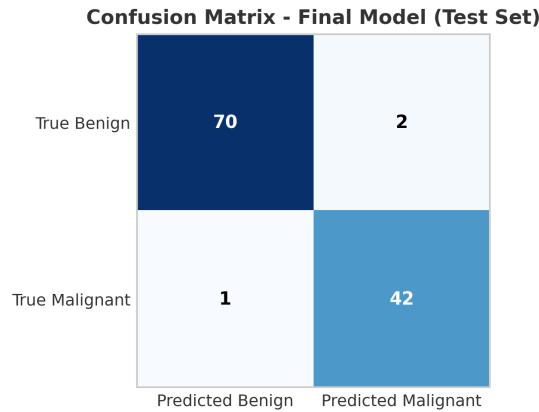


Figura 4: Matriz de confusão no conjunto de teste final (115 amostras).

Dois falsos positivos (benignos classificados como malignos) e um falso negativo. Sensibilidade para malignos:  $42/43 \approx 97,7\%$ ; especificidade:  $70/72 \approx 97,2\%$ .

**Rede rasa vs. profunda:** em val acc as três arquiteturas empatam (0,978), mas a profunda exige muito mais épocas para sair do platô (Figura 3) e apresenta maior train acc (0,995), sugerindo capacidade extra não aproveitada neste dataset tabular pequeno. Optamos pela rede [30,16,2] por parsimônia.

**Extras omitidos por brevidade:** no notebook também comparamos `MLPClassifier` (sklearn), com mesma test acc (97,4%), e avaliamos inicialização He e momentum (Seção 7), que encurtam a fase de latência; as figuras correspondentes ficam no repositório, mas foram omitidas aqui pelo limite de 6 páginas.

## 5 Interpretabilidade

Analizamos o `final_model` por gradientes de entrada e ablação. Também implementamos SHAP (Seção 9 do notebook); omitimos essa análise aqui por brevidade e limite de páginas. Os *summary plots* e *waterfalls* SHAP estão em `figures/shap_summary.png`.

### 5.1 Importância global

**Gradientes de entrada (6.1):** para cada amostra de teste, calculamos  $\partial P(\hat{c})/\partial x_j$  em relação à classe predita e médiamos  $|\partial P/\partial x_j|$ . A Figura 5 mostra que features `worst_*` ocupam as seis primeiras posições; `worst radius` e `worst texture` lideram ( $\approx 0,028$  e  $0,027$ ), seguidas de `worst perimeter`, `worst concave points` e `worst area`. Entre as `mean_*`, destacam-se `mean concave points`, `mean radius` e `mean perimeter`.

**Ablação (6.2):** zeramos features (individual, por família `mean_*/worst_*/*_error`, ou por atributo-base) e medimos aumento de cross-entropy. Com apenas 115 amostras de teste, queda de acurácia é **discreta** (passos de  $1/115 \approx 0,0087$ ), tornando `accuracy_drop` pouco discriminante. Ranqueamos por `ce_increase` (Figuras 6a a 6c).

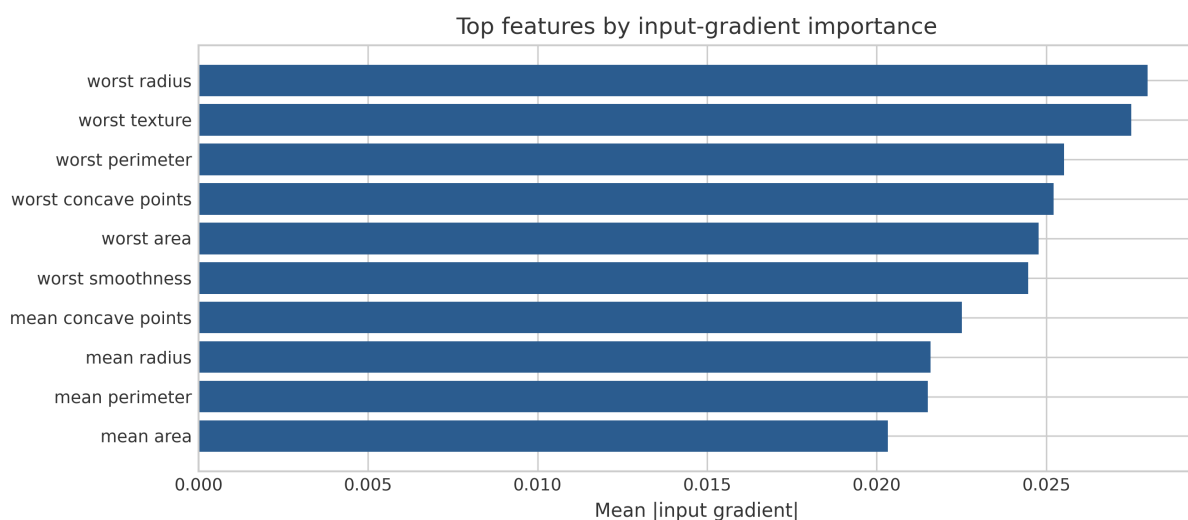


Figura 5: Top 10 features por importância global via gradiente de entrada ( $|\partial P/\partial x|$  médio no teste).

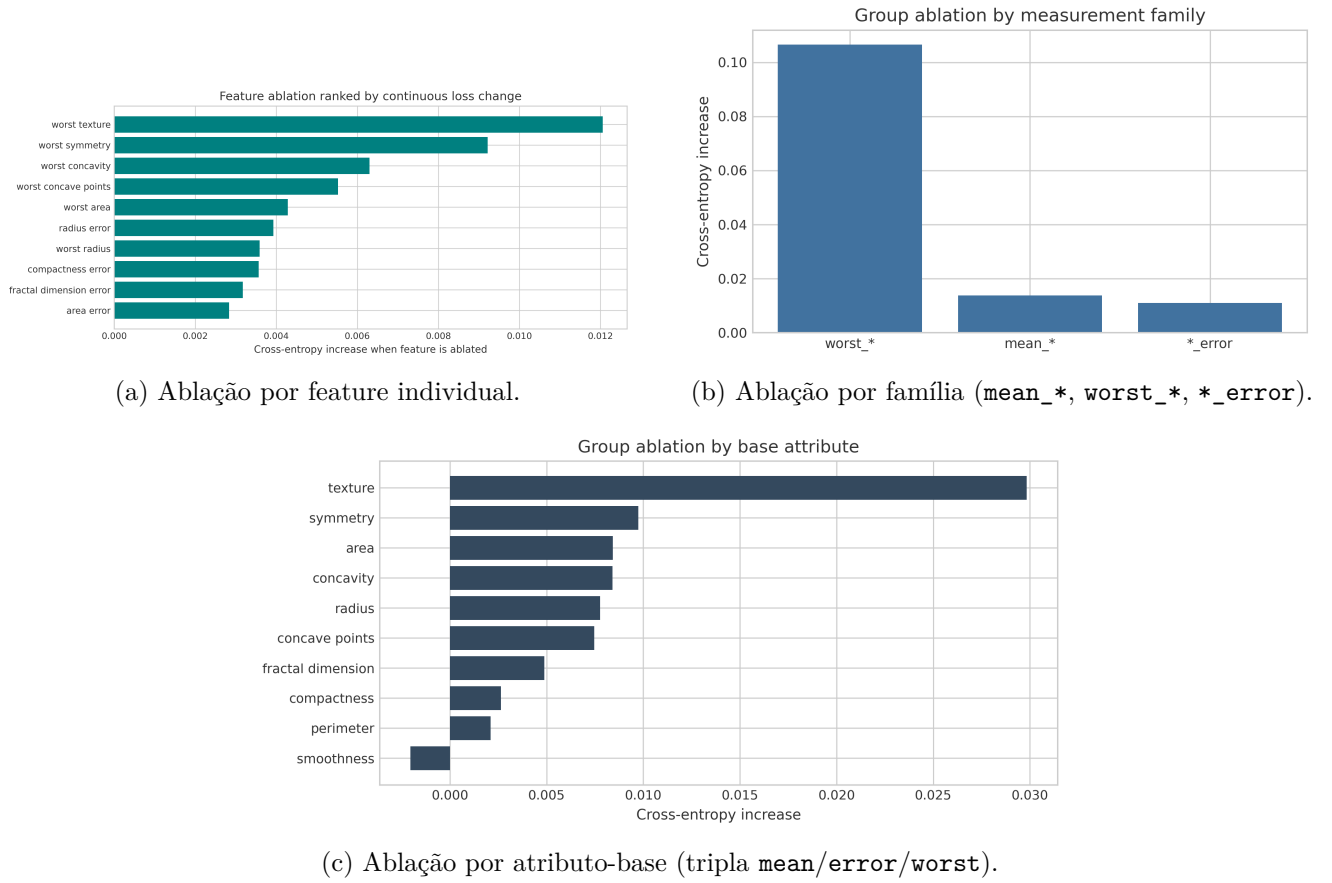


Figura 6: Ablação ranqueada por aumento de cross-entropy ao zerar grupos de features.

Na ablação por feature (Figura 6a), `worst texture` (+0,012 CE) e `worst symmetry` (+0,009) lideram. Por família (Figura 6b), remover todo o bloco `worst_*` eleva a CE em  $\approx 0,107$ , muito acima de `mean_*` (+0,014) e `*_error` (+0,011). Por atributo-base (Figura 6c), `texture` (+0,030) é o mais crítico; `symmetry`, `area` e `concavity` ficam empatados em torno de 0,008 a 0,010.

**Discordância entre métodos:** gradientes e ablação por feature coincidem parcialmente em `{worst area, worst concave points, worst radius, worst texture}`, mas a ablação por *atributo-base* destaca `texture` enquanto gradientes enfatizam morfologia de raio/perímetro. Essa divergência é esperada: gradientes medem sensibilidade local; ablação mede efeito global de remoção.

## 6 Discussão crítica

### Limitações metodológicas:

1. **Teste pequeno (115 amostras):** métricas de ablação por acurácia são grosseiras; preferimos CE contínua, mas incertezas permanecem elevadas.
2. **Validação estreita (91 amostras):** empates frequentes entre configs (val acc 0,978); escolhemos parsimônia, não superioridade estatística comprovada.
3. **Early stopping + retreino:** embora evitemos *peek* direto no teste durante busca de hiperparâmetros, o número de épocas ótimo foi estimado na validação; retreino em treino+val melhora robustez, mas não elimina variância de *seed*.
4. **Interpretabilidade:** gradientes pontuais e ablação binária (zerar feature) não capturam interações não-lineares; SHAP foi implementado mas omitido do relatório (ver notebook).

**O que faríamos diferente:** validação cruzada  $k$ -fold para seleção de hiperparâmetros; regularização L2 explícita na rede profunda; calibração de probabilidades; técnicas mais suaves (Integrated Gradients); e exportar as curvas de early stopping para o relatório.

**Conclusão:** a MLP manual atinge desempenho comparável ao `MLPClassifier`, com interpretabilidade consistente no domínio clínico. Medidas `worst_*` e morfologia de concavidade/textura são os preditores mais salientes, mas métodos de explicação devem ser lidos em conjunto, não isoladamente.

## Referências

- [1] Pedregosa et al. Scikit-learn: *Breast Cancer Wisconsin (Diagnostic) Dataset*. [https://scikit-learn.org/stable/datasets/toy\\_dataset.html#breast-cancer-dataset](https://scikit-learn.org/stable/datasets/toy_dataset.html#breast-cancer-dataset)